

Proposal #25

Remote Eye-Tracking *Midterm Report*

CS Capstone Team 12



PFW CS Department -Sponsor

Prof. Jay Johns -Faculty Advisor

Dominic Minnich -Team Leader

Sulaiman Hussain, Logan Smith, Kyle Benich -Team Members

Abstract

This project proposes the development of a software tool utilizing standard computer or phone cameras to perform eye-tracking for remote user experience (UX) testing. Unlike conventional methods that require specialized equipment and physical labs, this solution aims to provide a cost-effective and accessible alternative for UX researchers. The primary objectives are to enhance remote UX testing capabilities, improve data accuracy by capturing real-time eye movements, and increase the accessibility of UX testing to a broader audience. The project will focus on developing a Flask server with key functionalities, such as user authentication, project creation, eye-tracking, and video recording. The anticipated outcome is a fully functional tool that delivers detailed insights into user behavior remotely for specific web pages or designs.

Problem Statement

Context and Background | Traditional UX testing relies heavily on physical labs and expensive specialized hardware, limiting its accessibility and scalability. With the rise of remote work and digital products, there is a growing demand for remote UX testing solutions that provide the same level of detail and accuracy without the associated costs.

Problem Definition | The challenge is to create a software tool that enables remote, accurate, and accessible UX testing using only standard computer or phone cameras, thereby eliminating the need for specialized equipment.

Relevance | This project is vital as it addresses the current limitations of UX testing, democratizing access to detailed user behavior insights. By making UX testing more accessible, the tool will allow companies, researchers, and students to enhance digital product usability, ultimately leading to improved user experiences.

Project Goals & Objectives

SMART Goals | Develop and deploy a functional eye-tracking tool by May.
Achieve a minimum of 90% data accuracy rate in eye-tracking within six months.

Long-Term Objectives | The project aims to establish the software as a leading tool for remote UX testing, continually improve its features, and expand its user base possibly past PFW.

Short-Term Objectives | The immediate focus is on building core functionalities, conducting initial testing, and gathering feedback to refine the product for a more functional release.

Scope

In-Scope | The project will encompass the development of a Flask server, implementation of eye-tracking using computer or phone cameras, creation of user roles, and project management features, along with real-time video recording with eye-tracking overlays.

Out of Scope | The project will not include the development of specialized eye-tracking hardware or any analysis unrelated to user experience.

Constraints | Constraints include limited budget and resources, dependency on open-source tools and technologies, and time limitations due to academic deadlines.

Assumptions | The project assumes that users will have access to a standard webcam or phone camera and a stable internet connection for data transfer.

Methodology

Approach | The project will utilize an Agile development methodology, which allows for iterative building and refinement of the software tool based on user feedback and testing.

Tools and Technologies | The development stack includes Flask, Python, JavaScript, HTML, CSS, and SQLite for database management.

Data Sources | The project will leverage real-time eye-tracking data captured via webcams or phone cameras.

Development Process | The development process will involve several stages: planning, architecture design, core functionality development, eye-tracking integration, testing, and deployment.

Testing and Validation | Testing and validation will include unit tests, user acceptance testing (UAT), and scenario-based testing to ensure accuracy and reliability.

Timeline

Project Phases |

Planning: Ended October 17, 2024.

Development: Began October 19, 2024

Testing: ~

Deployment: ~

Milestones |

1. Completion of server setup.
2. Integration of eye-tracking functionality.

3. Initial user testing and feedback collection.
- 4.

Dependencies | The project relies on the availability of team members, access to hardware and software tools, and server stability.

Risk Analysis

Risk Identification | Potential risks include technical challenges with real-time data processing, privacy concerns regarding user data, and limited access to test users.

Risk Assessment | The technical challenges are of medium likelihood with high impact, while privacy concerns are of low likelihood but medium impact.

Mitigation Strategies | Regular code reviews, encrypted data handling, and clear consent protocols will be used to mitigate risks.

Stretch Goals

Mobile | Bring support for this web application to mobile in specific using the phones embedded camera for the project.

Heat Mapping | Extract heat maps from user eye tracking data on popular places the users are drawn to.

Domain Support | Support more than just the figma domain. This could be other established sites or custom sites.

Deliverables

Repository | Github repository containing version history, code sources, project timeline.

Documents | Documentation throughout our progress.

Functional Requirements

Feature List

The system must provide the following features:

- Users shall be able to create, edit, and delete their profiles, with the exception of "Guests", whose data will not be saved.
- Users shall have the capability to create new projects, including the addition of tasks, descriptions, and URLs.
- Users shall be able to share projects with other users or guests for analysis.
- The system shall perform real-time tracking of user eye movements using standard laptop webcams.
- The system shall record user sessions, including screen recordings with an overlay of eye-tracking data.
- Users shall have the option to log in, register, and select various roles, such as Analyst, Project Manager, and Admin.
- A user interface shall be provided for project creation, analysis, and viewing recorded sessions alongside data.

Data Handling

The system shall ensure secure and efficient data handling, including:

- Validation of input data for accuracy and completeness.
- Secure storage of user data, session recordings, and analysis results in the database.
- Encryption of all stored and transmitted data to protect user privacy.
- Data retrieval functionalities that support filtering and searching for analysis purposes based on the project.

System Actions

The system shall perform specific actions in response to user inputs or events:

- Generation of reports on user behavior based on eye-tracking data, eg) time taken to complete task.
- Requesting users to agree to our TOS and give certain browser based permissions.
- Notification of users regarding session status and progress.
- Display of real-time eye movement on the screen as a visual overlay.
- Authentication of users with access controls based on roles.

Integration Requirements

The system must integrate with external APIs and systems, including:

- Integration with third-party APIs and libraries, such as Webgazer.js, to enable eye-tracking functionality.
- Connection to external databases for data storage and retrieval if necessary.

User Roles and Permissions

The system shall define distinct roles and permissions for users:

- Admins shall manage all user accounts underneath them, projects, and system settings.
- Project Managers shall create projects, share them, and analyze results.
- Analysts shall have access to assigned projects and their analysis data.
- Guests shall have limited access, restricted to viewing shared projects without editing or creating capabilities.

Non-Functional Requirements

Performance Requirements

The system shall meet the following performance standards:

- Pages must load within 3 seconds under normal usage conditions.
- The system shall support real-time eye-tracking and screen recording without noticeable latency.

Security Requirements

The system must adhere to strict security protocols:

- AES-256 encryption shall be used for all data in transit and at rest.
- Multi-factor authentication shall be required for user login.

Usability Requirements

The system shall be user-friendly and accessible:

- Compliance with WCAG 2.1 Level AA accessibility guidelines is mandatory.
- The user interface shall be intuitive, with tooltips and help documentation readily available.

Reliability and Availability

The system shall ensure high reliability and availability:

- The system must maintain a high percentage uptime.
- Data backups shall occur every 24 hours to prevent loss.

Maintainability Requirements

The system shall be easy to maintain and update:

- The codebase shall be modular to facilitate updates and bug fixes.
- Version control shall be employed to manage changes and support collaboration.

Scalability Requirements

The system must support growth in users, data, and transactions:

- The system shall support concurrent users without performance degradation.
- The system shall efficiently handle large datasets, including eye-tracking data and video recordings.

Use Cases/User Stories

User Scenarios

Various scenarios describe how different users will interact with the system:

- As a registered user, I want to reset my password to regain access to my account.
- As a Project Manager, I want to create a new project and assign tasks to team members.
- As a Guest, I want to view shared projects to understand user behavior without needing an account.

Interaction Flows

The system shall provide detailed interaction flows for each user scenario:

- "Forgot Password" process: The user selects "Forgot Password," receives a reset link via email, and sets a new password.
- Admin management flow: Admin navigates to the user management panel to add or remove users.
- Project sharing flow: The Project Manager shares a project link, and a Guest user accesses the link, views the project, and starts analysis.

User Roles and Goals

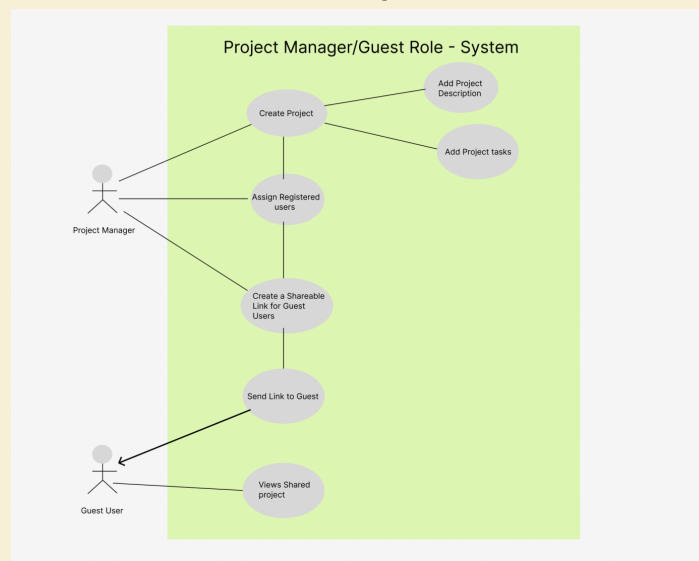
The system shall support the following user roles and their objectives:

- Admin users manage system-wide settings, user accounts, and data integrity.
- Project Managers create, manage, and analyze UX testing projects.
- Student users engage with assigned projects, perform testing, and review their analysis.
- Guest users access and view shared projects for feedback purposes.

Success and Failure Conditions

The system shall define conditions for successful and failed interactions:

- Success: The user successfully resets their password and logs in.
- Failure: The user receives an error message for an invalid reset link.



User Interface Requirements

Design Principles

The user interface shall adhere to the following design standards:

- A consistent color scheme and typography shall be used throughout the interface.
- The design shall prioritize simplicity and ease of navigation.

Wireframes

The system shall include wireframes or mockups to represent the user interface visually:

- The login page shall include fields for username and password, a "Login" button, and a "Forgot Password" link.
- The project creation page shall provide fields for the project title, description, tasks, and sharing options.

Navigation Requirements

The system shall provide clear navigation paths:

- The main navigation menu shall be accessible from all pages and include links to Home, Profile, Settings, and Logout.
- Breadcrumb navigation shall be implemented to enhance usability.

Accessibility Features

The system shall ensure accessibility for all users:

- All interactive elements must be keyboard accessible, and alternative text shall be provided for all images.

Sprints

Sprint 0 (Ended Sunday, October 17)

Goals: Large planning sprint.

Accomplished: Planned out the entire project.

Sprint 1 (Ended Sunday, October 27)

Goals: Set up Flask Server.

Accomplished: Established Flask server with route conventions and user authentication.

1 Flask server Setup.

2 Login Register pages set up and link to db population (initial).

3 User Authentication established via cookies.

Comments: For our first sprint, we decided to set up the backbone of our design.

We know that one of our project goals is to integrate our application into a Proxmox server later on, but we wanted to get started with something before we integrate with it.

We first wanted to integrate a Flask server to get us started and allow us to begin development before integrating with our sponsor's server.

For our first sprint, we were able to do just that.

We got Flask up and running and began working on the structure of our design.

In order to test the Flask server functionality, we set up our login and register pages.

We also made sure to organize it so we could begin working on the different web pages necessary for the application.

Sprint 2 (Ends Sunday, Nov 3)

Goals: Establish Individual Pages (Front).

Accomplished: Began Implementing.

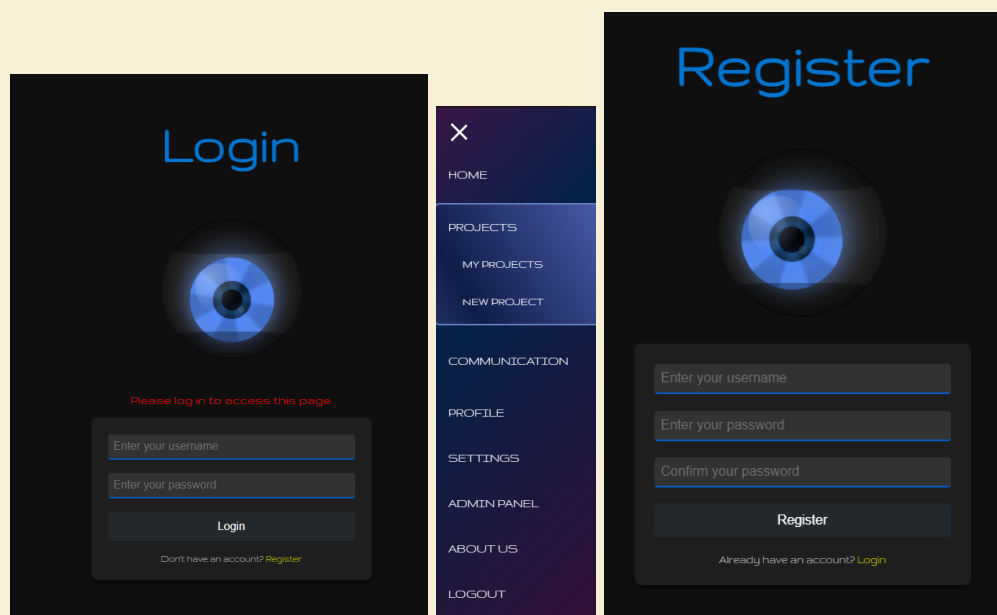
1 Fluid timeline established.

2 Front-end page work posted (individual).

3 Individual pages with corresponding routes were created.

Comments: In our second sprint, we decided to divide and conquer. We all wanted a part in the development as full stack developers, and this is the point we get to do just that. We split up the different pages necessary for the application amongst the group in an effort to get hands-on experience in front-end and back-end development. We chose to work on the Create, View and Edit portion of the application. We strategically chose these pages so we would have the chance to work together, but also separately engage in front-end and back-end development.

During this process, we were each able to get set up and get started. We are still in the process of developing these pages and how they interact with the flask server. During this sprint, we were all able to access GitHub and get our project setup for all of us to interact with. There were some minor issues learning how to navigate the project and get everything established to do our work. In the end, we were able to get all these concerns addressed and we have each begun working on these pages. We plan on completing this sprint by Sunday.

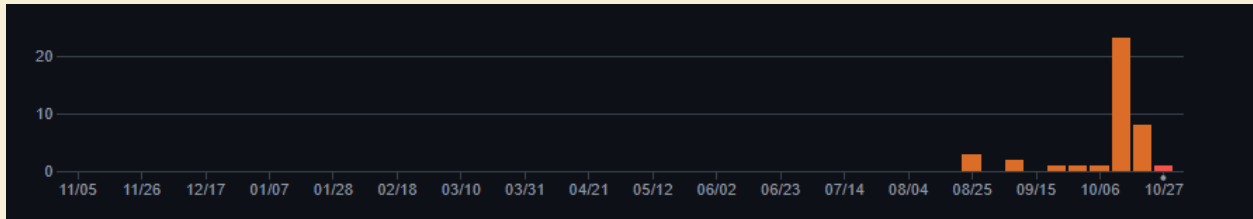


Sprint 3 (Begins Tuesday, Nov 5)

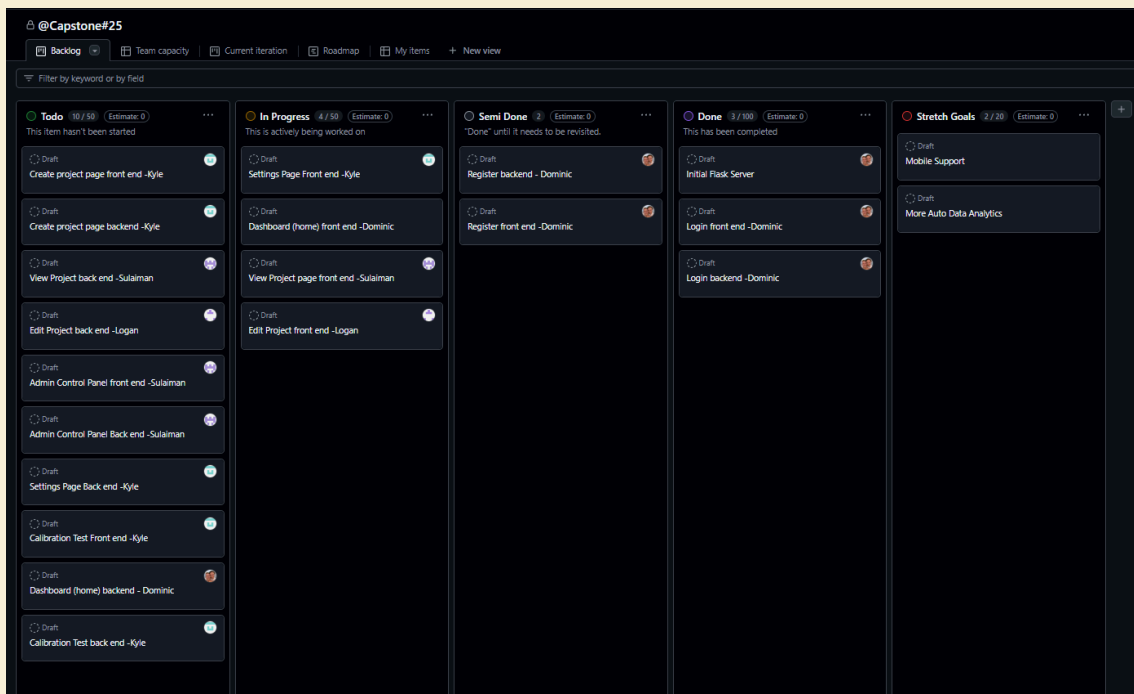
Goals: Establish functional backend tasks to be worked on during sprint 3

Accomplished: NA

Github



Code Comments



GitHub Projects

Glossary

Admin

A user role with the highest permissions, responsible for managing user accounts, projects, and system settings.

Analyst

A user role focused on analyzing project data and providing insights, typically without permissions to create or manage projects.

AES-256 Encryption

A secure encryption standard used for protecting data at rest and in transit, ensuring the confidentiality of user information.

Breadcrumb Navigation

A user interface feature that displays the user's current location within a web application and provides links to previous sections for easy navigation.

Computer Vision

Technology used for real-time tracking of eye movements through standard webcams, analyzing gaze points and behavior without specialized hardware.

Data Encryption

The process of encoding data to prevent unauthorized access, crucial for ensuring user privacy and compliance with security protocols.

Eye-Tracking

A method for tracking and analyzing eye movements to study user interactions with digital interfaces; in this project, it is implemented via standard cameras.

Figma Domain

A reference to the Figma design platform; the project initially supports eye-tracking analysis specifically on this domain.

Flask

A lightweight web application framework used to develop the server backend for this eye-tracking tool.

Guest User

A limited-access user role that allows viewing shared projects without the ability to edit or create content.

Heat Mapping

A visual representation of areas where users' attention is focused on a screen, often used to analyze popular gaze points.

JavaScript, HTML, CSS

Core technologies used to build the front-end of the application, providing structure, style, and interactivity.

Mobile Support

Extending application capabilities to mobile devices, using embedded cameras for eye-tracking.

Multi-Factor Authentication (MFA)

A security feature requiring multiple forms of verification before granting access, enhancing login security.

Privacy Consent

A protocol where users agree to terms of service and grant necessary permissions, ensuring legal compliance for data collection.

Project Manager

A user role responsible for creating, sharing, and analyzing UX testing projects within the system.

Real-Time Eye-Tracking Overlay

A feature that visually represents users' gaze points on the screen during live sessions, aiding UX analysis.

Repository (GitHub)

A version-controlled codebase storage, which includes the project's history, source code, and documentation.

Role-Based Access Control (RBAC)

A system for defining permissions based on user roles, such as Admin, Project Manager, Analyst, and Guest, to maintain structured access.

Screen Recording

The process of capturing video of the user's screen activity along with eye-tracking data for analysis.

SQLite

A lightweight database used for managing and storing user data, project data, and analysis results.

Standard Webcam or Phone Camera

The hardware requirement for the application's eye-tracking feature, providing accessibility without specialized devices.

Success and Failure Conditions

Criteria defining successful or unsuccessful outcomes for specific system interactions, like password resets.

Testing and Validation

Processes including unit tests, user acceptance testing, and scenario-based testing to ensure software reliability and performance.

User Acceptance Testing (UAT)

A testing phase where end-users validate the software's functionality and usability before full deployment.

User Behavior Insights

Data-driven findings about user interaction patterns, aiding in UX optimization and design improvements.

User Experience (UX)

The overall experience and satisfaction a user has when interacting with a digital product or system, which this tool aims to enhance through eye-tracking.

UX Testing

The process of evaluating how users interact with a product to identify usability issues, improve functionality, and enhance the overall user experience.

Webgazer.js

A JavaScript library used for implementing eye-tracking features through browsers, capturing gaze data without specialized hardware.

WCAG 2.1 Level AA Accessibility Guidelines

Web Content Accessibility Guidelines ensuring the application is accessible to all users, including those with disabilities.

Wireframes

Preliminary visual representations of the application's user interface, guiding design decisions for usability.